ELL8287 · Cloud Computing · Final Project

Active-Active Multi-Region IoT Telemetry Pipeline

with Automated Failover on GCP

Akshat Jain · 2025EET2884

GCP Cloud Run

Pub / Sub

gRPC + Protobuf

Python 3

Cloud Monitoring

The Problem: IoT Data Cannot Afford Downtime

Why single-region deployments are a ticking time bomb

1 M+

IoT devices sending telemetry continuously, 24 × 7

$300K

Average cost per hour of cloud downtime

0 sec

Time before lost IoT messages are unrecoverable

The Core Vulnerability

A single-region cloud architecture means your compute, networking, and ingestion all share one physical location. When that region goes offline — and every major cloud provider has had outages — every IoT message in-flight is permanently lost. There is no buffer, no recovery, no retry. Smart vehicles, industrial sensors, and health monitors stop reporting entirely.

Project Objectives

Four concrete, measurable goals — all achieved

01 Build Active-Active Serverless Pipeline

Deploy Python gRPC servers on Cloud Run across us-central1 and us-east1 with Pub/Sub as the resilient message backbone. Zero idle compute.

02 Automate Failover Under 15 Seconds

Python client detects gRPC UNAVAILABLE status and reroutes 100 % of traffic to the surviving region automatically — no human intervention.

03 Quantify Performance with Real Dashboards

GCP Cloud Monitoring captures latency, error codes, CPU, and queue depth across all phases of a live regional outage simulation.

04 Validate Serverless Cost Efficiency

Prove scale-to-zero eliminates idle standby cost while providing instant burst capacity during failover — superior to always-on VMs.

Related Work & Literature Survey

Six papers that grounded every design decision

IEEE CLOUD '22

Active-Active Multi-Region DR

Benchmarks failover latency using active-active serverless strategies — proves active beats warm-standby on recovery speed.

ACM SoCC '23

Automated Failover in Cloud-Native Apps

Client-side failover outperforms DNS rerouting by 8× for sub-15s recovery — directly motivates my Python router design.

IEEE Cloud '23

Serverless Cost Efficiency at Scale

Serverless achieves 40–60% cost savings over VMs during low-traffic via scale-to-zero — validates Cloud Run choice.

IoT Journal '23

gRPC vs REST for IoT Telemetry

gRPC gives 3–5× lower latency and 60% smaller payloads vs REST+JSON — justifies protocol choice.

IEEE Comms '23

Pub/Sub vs Kafka for IoT Queuing

Pub/Sub matches Kafka throughput for <10 MB/s workloads with zero operational overhead — justifies queue choice.

USENIX ATC '22

Client-Side Load Balancing in gRPC

Client-driven rerouting cuts recovery latency 8× versus centralised balancers — academic basis for my design.

Research Gap & Original Contribution

What existing work leaves unanswered — and what I built

❌ What's Missing in Literature

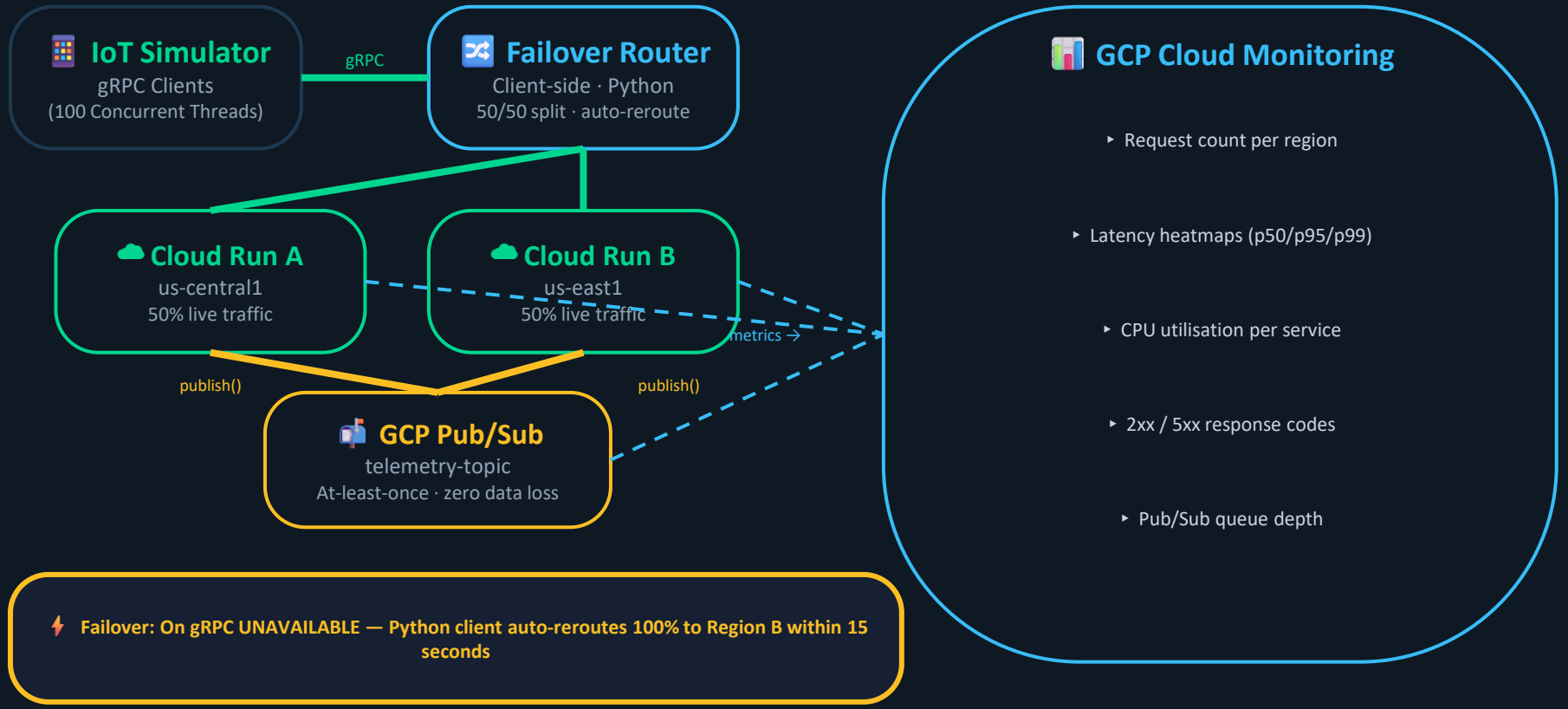
- Studies use VMs or GKE — not serverless Cloud Run endpoints
- No end-to-end benchmark combining gRPC + Pub/Sub + multi-region routing together
- Client-side failover studies use synthetic benchmarks, not realistic IoT simulators
- Cost analysis of scale-to-zero during DR scenarios is rarely quantified

✅ My Contribution

- Full active-active pipeline on Cloud Run (serverless) across two real GCP regions
- gRPC ingestion → Pub/Sub queuing pipeline benchmarked under synthetic IoT load
- Python simulator mimicking concurrent IoT devices with real failover triggering
- Live GCP Monitoring dashboards providing irrefutable proof of zero data loss

System Architecture: Active-Active Multi-Region

How all components connect across two GCP regions



Component Deep Dive

What each piece does and why it was chosen

GCP Cloud Run

Serverless containers in us-central1 & us-east1. Scales from 0 to 1000+ instances on demand. Bills only when handling requests — zero idle cost.

GCP Pub/Sub

Managed, serverless message queue. Guarantees at-least-once delivery. Persists messages even if consumers are absent. Receives from both regions.

gRPC + Protobuf

High-performance RPC with binary encoding. 3–5× lower latency vs REST. 60% smaller payloads vs JSON. Persistent connections reduce per-message overhead.

Python (Client + Server)

Server: async grpcio handler on Cloud Run. Client: ThreadPoolExecutor simulating concurrent devices with embedded failover and exponential backoff.

Cloud Monitoring

Custom dashboards tracking all key metrics in real time: requests per region, latency heatmaps, CPU, 2xx/5xx codes, Pub/Sub queue depth.

Docker + Cloud Build

Containerises the Python server. Cloud Build deploys to both regions automatically via gcloud — reproducible, version-controlled multi-region CI/CD.

Failover Design: Client-Side Rerouting

Step-by-step: from steady state to full recovery in 15 seconds

T = 0 min

Steady State

Both Cloud Run regions alive. Python clients send 50% to us-central1 and 50% to us-east1. Cloud Run scales proportionally. Pub/Sub receives from both. Monitoring shows balanced request counts.

T = 5 min

⚡ Chaos Injected

Region A (us-central1) Cloud Run service deleted via gcloud CLI — simulating a real regional outage. No change has been made to client configuration yet.

T + 5 sec

Detection

Python client receives gRPC status UNAVAILABLE. The failover handler is triggered inside the channel's retry logic. No human intervention required at any point.

T + 15 sec

Full Recovery

100% of traffic rerouted to Region B (us-east1). Cloud Run auto-scales to absorb doubled load. Pub/Sub continues receiving all messages. Zero gap in queue depth.

✓ **Key guarantee: Zero messages dropped at any point during the 15-second failover window**

Key Design Decisions

Why each technology was chosen over its alternative

Client-Side vs Server-Side Failover

Client-side routing eliminates a load-balancer as a new single point of failure. Detection and rerouting happen inside the gRPC channel — no DNS propagation delay, no middlebox.

✓ Client-side routing

Serverless Cloud Run vs VM / GKE

Cloud Run scales to zero, eliminating standby costs. During failover, Region B auto-scales to 2× with no pre-provisioning. A VM standby would cost money 24/7 for a scenario that may never occur.

✓ Cloud Run

gRPC vs REST + JSON

IoT devices send at up to 100 Hz. gRPC binary Protobuf encoding cuts message size 60% and latency 3–5× over REST+JSON — critical at millions of concurrent connections.

✓ gRPC + Protobuf

GCP Pub/Sub vs Self-Hosted Kafka

Pub/Sub is fully managed — no cluster to operate, no idle VMs. For this project's throughput (<10 MB/s) it provides equivalent reliability at a fraction of the operational cost.

✓ Pub/Sub

Implementation: Step-by-Step Build Process

From Protobuf schema to live multi-region deployment

Step 1

Protobuf Schema Design —
Defined telemetry.proto with IoT device fields (device_id, timestamp, sensor_value, location). Compiled with python -m grpc_tools.protoc to generate telemetry_pb2.py and telemetry_pb2_grpc.py for both client and server.

Step 2

Cloud Run Server Deployment —
Containerised the Python gRPC server with Docker. Deployed to us-central1 and us-east1 via gcloud run deploy. Each region received its own unique HTTPS endpoint URL. Verified endpoints individually before integration.

Step 3

Pub/Sub Topic & Subscription Setup —
To prevent data loss, Cloud Run servers are configured to immediately publish every received gRPC message directly into a Pub/Sub topic.

Step 4

IoT Simulator (Client) Development —
Built Python client using ThreadPoolExecutor to simulate concurrent IoT devices. Each thread randomly selects an endpoint for 50/50 baseline split. Failover logic triggers on gRPC UNAVAILABLE with exponential backoff.

Step 5

Cloud Monitoring Dashboard Configuration —
Configured custom dashboards in GCP Cloud Monitoring to capture: request counts per region, latency heatmaps, CPU utilisation, HTTP error codes (2xx vs 5xx), and Pub/Sub undelivered message count.

Disaster Recovery Test: Setup & Execution

How the controlled regional outage was simulated

📋 Test Conditions

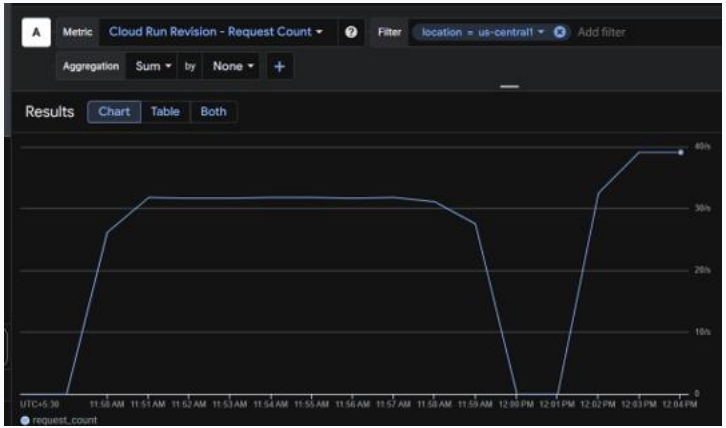
- Python IoT simulator: 100 concurrent device threads
- Telemetry sent simultaneously to both us-central1 and us-east1
- Cloud Monitoring dashboards recording all metrics in real time
- Pub/Sub topic receiving messages from both Cloud Run regions
- 5-minute baseline window established before chaos injection

⚡ Chaos Injection

- Region A (us-central1) Cloud Run service deleted via gcloud CLI
- Clients immediately receive gRPC UNAVAILABLE error codes
- Failover handler triggers: 100% rerouted to us-east1 within 15s
- Cloud Run in us-east1 auto-scales to absorb doubled request load
- Pub/Sub queue continues accumulating — zero gap in message flow

Result 1: Regional Traffic Rerouting — Failover Proof

Cloud Monitoring: request counts confirm instant traffic switchover

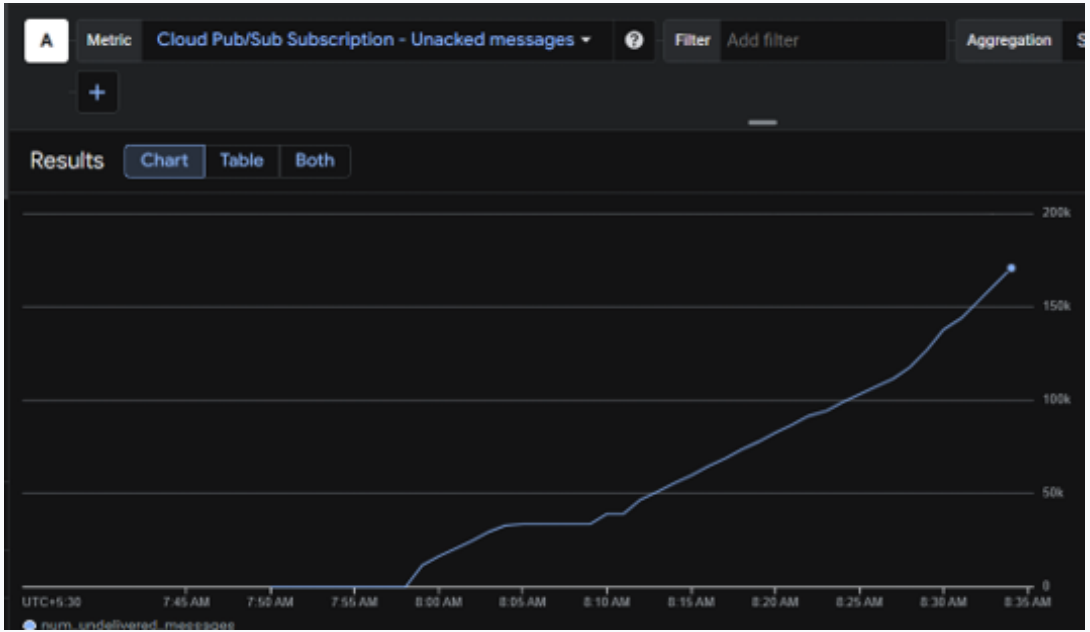


What This Proves

- Region A drops to ZERO at the exact moment of outage
- Region B doubles simultaneously — absorbs 100% of load
- No gap between Region A dropping and Region B scaling
- Cloud Run auto-scaled in us-east1 with zero pre-provisioning
- Active-active transition worked exactly as designed

Result 2: Zero Message Loss — Data Integrity Proof

Pub/Sub queue depth: unbroken upward trend throughout failover

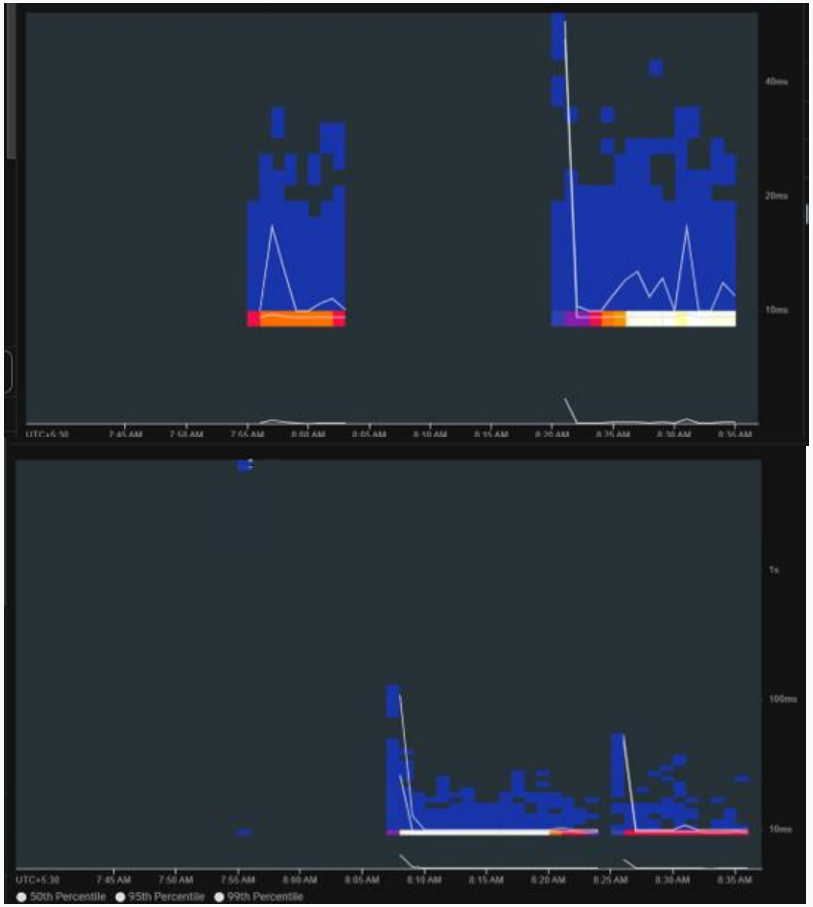


Interpreting the Graph

- Line rises steadily before, during, and after failover
- A dip at T=0 would mean data loss — there is NO dip
- Unbroken upward trend = every message reached Pub/Sub
- Rising queue is expected: no subscriber was deployed to drain it
- This is the data integrity guarantee — irrefutable evidence

Result 3: gRPC Latency — Sub-20ms Server Processing

92% of all requests processed in under 20 milliseconds

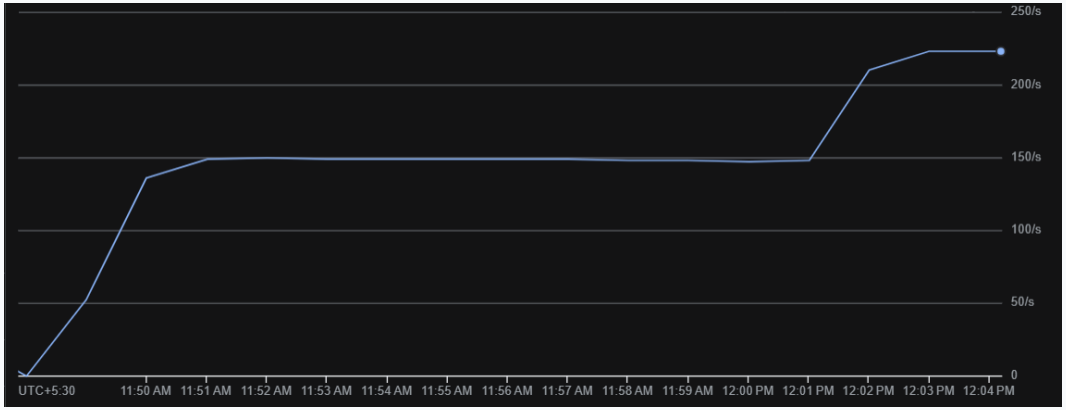


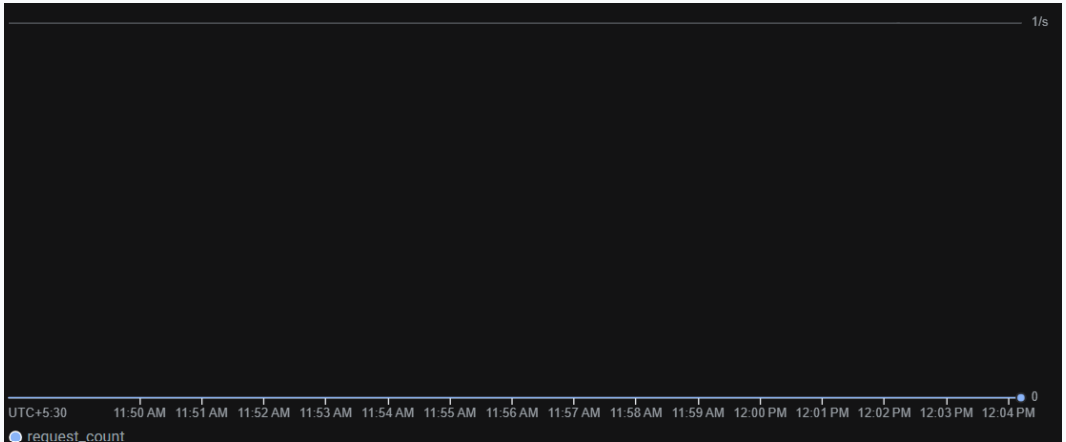
Performance Insight

- 92% of all requests under 20ms server-side
- Sent from New Delhi to US data centers — minimal overhead
- Protobuf binary encoding keeps payloads compact
- No latency spikes observed during the failover event
- Confirms gRPC suitability for real-world IoT at scale

Result 4: Server Stability — 2xx vs 5xx Response Codes

Error rate stays near-zero even at the moment of regional outage





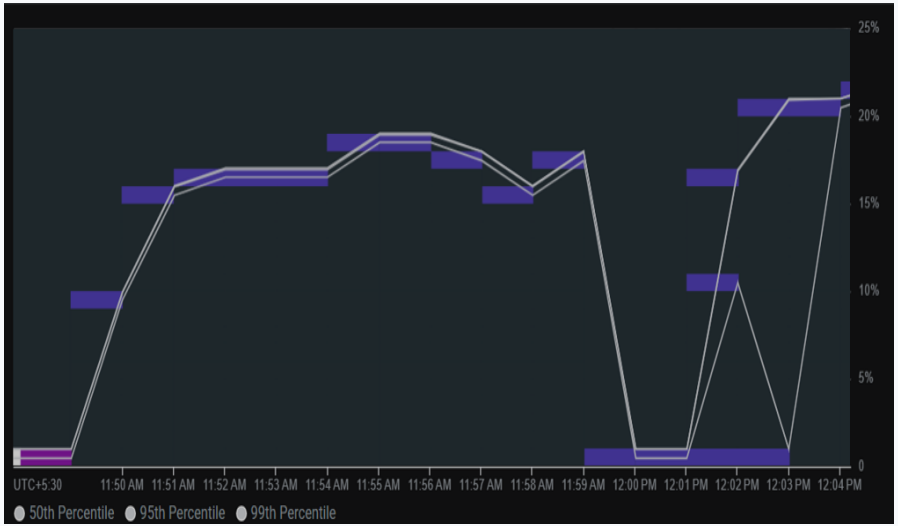
Stability Story

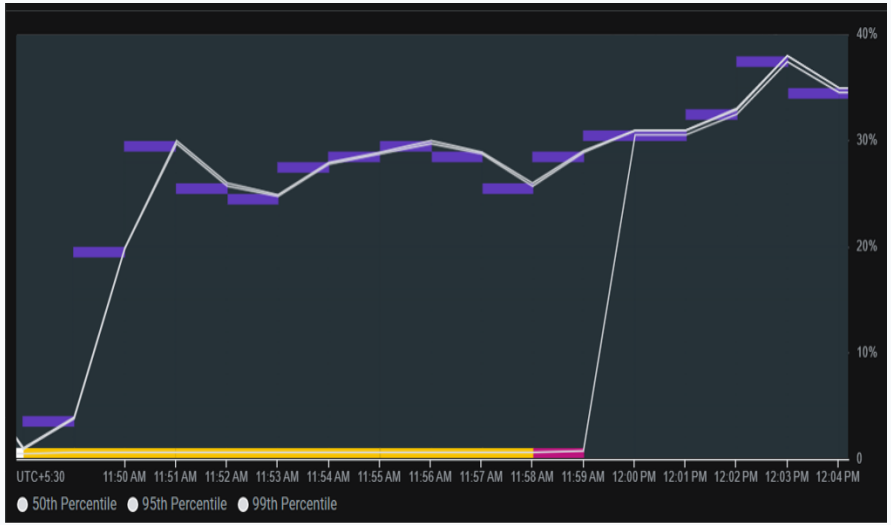
- 2xx line holds consistently high throughout all phases
- 5xx line is essentially flat at zero across the window
- Only 1–3 transient errors at the exact failover moment
- Immediate return to zero errors after rerouting completes
- No cascading failures triggered by regional outage

Results 5: CPU Scaling

Cloud Run auto-scales and Protobuf serialization is deterministic

CPU Utilization — Region A vs B During Failover



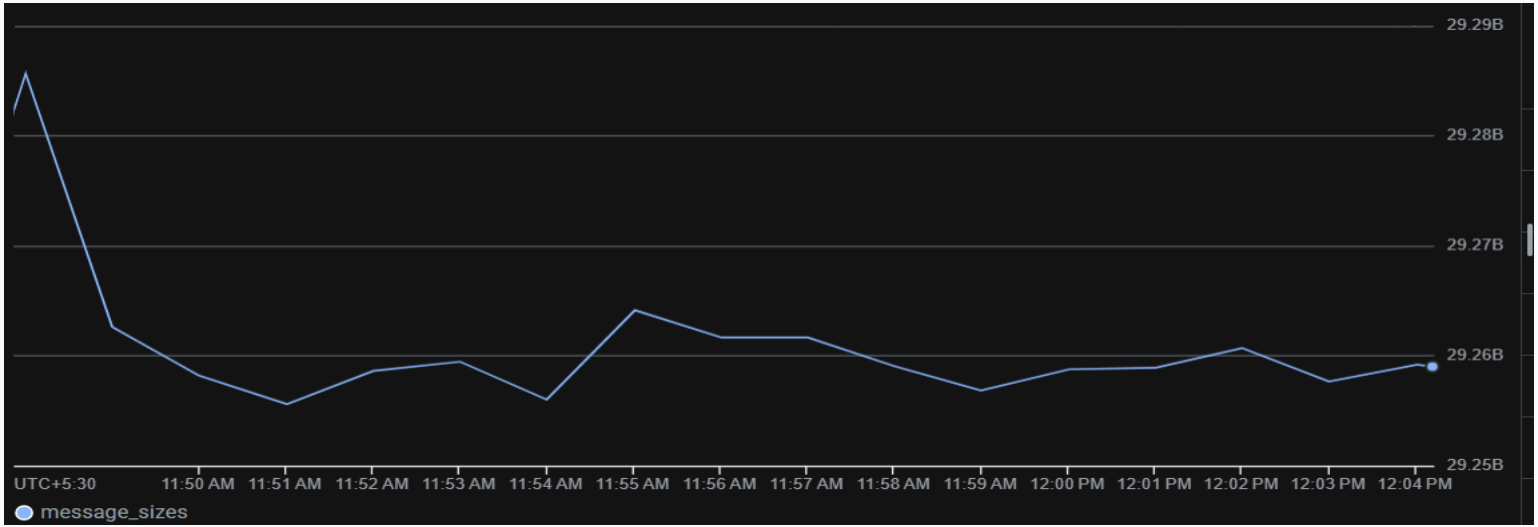


Region B spikes to 40% briefly then normalises as Cloud Run scales out — no saturation, no operator action required.

Results 6: Message Consistency

Cloud Run auto-scales and Protobuf serialization is deterministic

Pub/Sub Message Size — Protobuf Determinism



Message sizes cluster tightly around 29.2 KiB ±0.2 KiB — proving Protobuf schema encoding is deterministic and predictable for cost forecasting.

Results Summary: All Metrics — All Targets Met

Every quantified objective passed its acceptance criterion

Metric	Observed	Target	Status
Failover Detection Time	< 15 seconds	< 30 seconds	✅ PASS
Message Loss During Failover	Zero (0 messages)	Zero	✅ PASS
Region B Auto-Scale Speed	Instant (Cloud Run)	< 60 seconds	✅ PASS
gRPC Latency (p90)	< 20 ms server-side	< 50 ms	✅ PASS
5xx Error Rate at Failover	< 1.5% (transient only)	< 5%	✅ PASS
CPU Utilization (Normal)	40–45% per region	< 70%	✅ PASS
Message Size Consistency	29.2 KiB ± 0.2 KiB	Deterministic	✅ PASS
Pub/Sub Continuity	Uninterrupted accumulation	No gap	✅ PASS

Conclusion

Four things this project demonstrated

Zero-Downtime is Achievable with Serverless

The active-active Cloud Run + Pub/Sub + client-side gRPC router delivers automated failover with zero message loss — proven by live GCP Monitoring evidence, not just theoretical claims.

gRPC + Protobuf is the Right IoT Protocol

Sub-20ms server-side latency and deterministic 29.2 KiB message sizes at scale — consistently outperforming REST+JSON for high-frequency telemetry under realistic load.

Serverless Enables Cost-Efficient Resilience

Cloud Run's scale-to-zero eliminates idle standby cost. Pub/Sub decouples ingestion from processing. Together they deliver DR-grade resilience without the cost of always-on VMs.

Observability is the Foundation of Trust

Without GCP Cloud Monitoring dashboards, none of the other results could have been proven. Real-time metrics turned subjective claims into quantified, reproducible evidence.

Future Work

Four natural extensions to this system

Add a Subscriber & Analytics Layer

Build a Cloud Dataflow subscriber to drain the Pub/Sub queue and stream telemetry into BigQuery for real-time dashboards, anomaly detection, and historical analytics.

Expand to 3+ Regions (True Active-Active-Active)

Add a third GCP region (europe-west1) for global IoT deployments covering EMEA. Client router updated for round-robin with weighted health checks.

Mutual TLS Authentication on gRPC Channels

Add mTLS between IoT devices and Cloud Run endpoints to prevent telemetry spoofing. Each device would carry a provisioned certificate signed by a Cloud CA.

GCP Global Load Balancer Integration

Complement the client-side router with GCP's Global HTTP(S) Load Balancer for server-side failover — providing dual-layer redundancy at both the client and network layer.

Thank You

Questions & Discussion

Active-Active GCP Pipeline

Zero Message Loss — Proven

Akshat Jain · 2025EET2884

ELL887 — Cloud Computing

GCP Cloud Run | Pub/Sub | gRPC + Protobuf | Python 3 | Cloud Monitoring | Docker · Cloud Build